Android Navigation with Jetpack Compose

Mobile Application Development

Ir. Aryo Pinandito, Ph.D.

Android Navigation

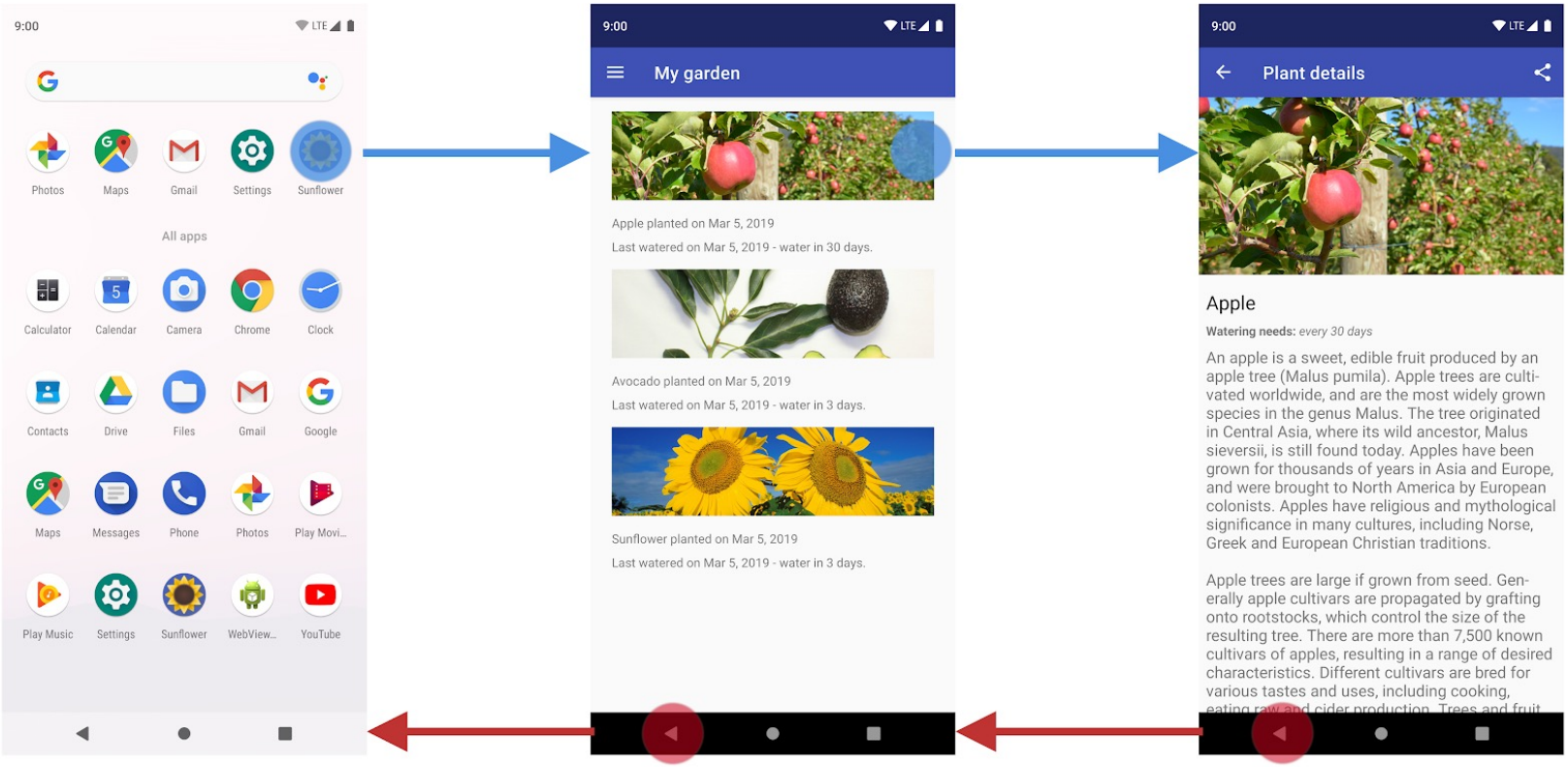
Navigation User Experience

- Navigation between different screens and apps is a core part of the user experience.
- The Navigation component is designed to implement these principles by default, ensuring that users can apply the same heuristics and patterns in navigation as users move between apps.

Fixed Start Destination

- Every app has a fixed start destination.
- This is the first screen the user sees when they launch an app from the launcher.
- This destination is also the last screen the user sees when they return to the launcher after pressing the Back button.

Fixed Start Destination



Launcher **List Screen** **Detail Screen**

Navigation State as A Stack

- When an app is first launched, a new task is created, and app displays its start destination.
  - This becomes the base destination of what is known as the back stack and is the basis for your app's navigation state.
- The top of the stack is the current screen, and the previous destinations in the stack represent the history of where the users have been.
- The back stack always has the start destination of the app at the bottom of the stack.

Navigation State as A Stack

- Operations that change the back stack always operate on the top of the stack, either by pushing a new destination onto the top of the stack or popping the top-most destination off the stack.
- Navigating to a destination pushes that destination on top of the stack.
- The Navigation component manages all of the back stack ordering, though an app could manage the back stack themself.

Up and Back are Identical

- The Back button appears in the system navigation bar at the bottom of the screen and is used to navigate in reverse-chronological order through the history of screens the user has recently worked with.
- The Up button appears in the app bar at the top of the screen but never exits an app.

The Up and Back buttons behave identically.

Navigation with Jetpack Compose

Navigation with Compose

- The Navigation component provides support for Jetpack Compose applications.
- Assist navigation between composables while taking advantage of the Navigation component's infrastructure and features.

Dependencies

To support Compose, use the following dependency in app module's build.gradle file:

```
dependencies {
  ...
  implementation("androidx.navigation:navigation-compose:2.9.4")
  ...
}
```

Navigation

- Navigation refers to the interactions that let users navigate across, into, and back out from the different pieces of content within an Android app.
- Android Jetpack's Navigation component includes the Navigation library, Safe Args Gradle plug-in, and tooling to help developers implement app navigation.

Navigation

- The Navigation component handles diverse navigation use cases, from straightforward button clicks to more complex patterns, such as app bars and the navigation drawer.

Key Concepts in Navigation

- Host (NavHost)
  - A UI element that contains the current navigation destination. That is, when a user navigates through an app, the app essentially swaps destinations in and out of the navigation host.
- Graph (NavGraph)
  - A data structure that defines all the navigation destinations within the app and how they connect together.

Key Concepts in Navigation

- Controller (NavController)
  - The central coordinator for managing navigation between destinations. The controller offers methods for navigating between destinations, handling deep links, managing the back stack, and more.
- Destination (NavDestination)
  - A node in the navigation graph. When the user navigates to this node, the host displays its content.

Key Concepts in Navigation

- Route
  - Uniquely identifies a destination and any data required by it.
  - App navigates using routes. Routes take users to destinations.
  - It can be any serializable data type.

Benefits and Features

- Animations and transitions
- Deep linking
- UI patterns
- Type safety
- ViewModel support
- Fragment transactions
- Back and up

```
plugins {
  // Kotlin serialization plugin for type safe routes and navigation arguments
  kotlin("plugin.serialization") version "2.0.21"
}

dependencies {
  val nav_version = "2.9.4"

  // Jetpack Compose integration
  implementation("androidx.navigation:navigation-compose:$nav_version")

  // Views/Fragments integration
  implementation("androidx.navigation:navigation-fragment:$nav_version")
  implementation("androidx.navigation:navigation-ui:$nav_version")

  // Feature module support for Fragments
  implementation("androidx.navigation:navigation-dynamic-features-fragment:$nav_version")

  // Testing Navigation
  androidTestImplementation("androidx.navigation:navigation-testing:$nav_version")

  // JSON serialization library, works with the Kotlin serialization plugin
  implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
}
```

To include navigation support in Android Studio project, add the following dependencies to app's build.gradle file:

NavController

- NavController is the central navigation API.
- It tracks which destinations the user has visited, and allows the user to move between destinations.
- To create a NavController when using Jetpack Compose, call:

```
val navController = rememberNavController()
```

Up, Up, Up, High

- You should create the NavController high in your composable hierarchy.
  - High enough that all the composables that need to reference it can do so.
- Lets developers use NavController as the single source of truth for updating composables outside of your screens.
  - This follows the principles of state hoisting.

State Hoisting

- State hoisting in Compose is a pattern of moving state to a composable's caller to make a composable stateless.
- The general pattern for state hoisting in Jetpack Compose is to replace the state variable with two parameters:
  - **`value: T`**: the current value to display
  - **`onValueChange: (T) -> Unit`**: an event that requests the value to change, where T is the proposed new value

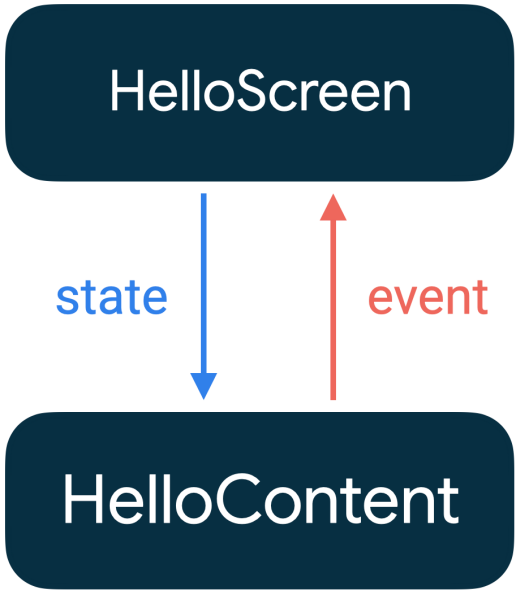
State Hoisting

State that is hoisted has some important properties/benefits:

- **Single source of truth**
- **Encapsulated**
- **Shareable**
- **Interceptable**
- **Decoupled**

```
@Composable
fun HelloScreen() {
  var name by
    rememberSaveable { mutableStateOf("") }
  HelloContent(name = name,
    onNameChange = { name = it })
}

@Composable
fun HelloContent(name: String,
  onNameChange: (String) -> Unit) {
    Column {
      Text(
        text = "Hello, $name",
      )
      OutlinedTextField(value = name,
        onValueChange = onNameChange,
        label = { Text("Name") })
    }
}
```



The pattern where the state goes down, and events go up is called a unidirectional data flow.

In this case, the state goes down from HelloScreen to HelloContent and events go up from HelloContent to HelloScreen.

Compose

- A serializable object or class is used to define a *route*.
- A route describes how to get to a destination, and contains all the information that the destination requires.
- Use the @Serializable annotation to automatically create the necessary serialization and deserialization methods for your route types.

@Serializable annotation is provided by the Kotlin Serialization plugin.

```
plugins {
  // Kotlin serialization plugin
  // for type safe routes and navigation arguments
  kotlin("plugin.serialization") version "2.0.21"
}

dependencies {
  // for @Serializable annotation
  implementation("org.jetbrains.kotlinx:
    kotlinx-serialization-json:1.7.3")
}
```

```
1   @Serializable
2   object Profile
3   @Serializable
4   object FriendsList
5
6   val navController = rememberNavController()
7
8   NavHost(navController = navController,
9     startDestination = Profile) {
10      composable<Profile> { ProfileScreen( ... ) }
11      composable<FriendsList> { FriendsListScreen( ... ) }
12      // Add more destinations similarly.
13  }
```

Define (serializable) objects as routes

Create NavController

Create NavHost

Composable to navigate as Lambda's

Pass Arguments

If passing data to a destination is needed, define the route with a class that has parameters.

```
@Serializable
data class Profile(val name: String)

@Serializable // with nullable fields
data class Profile(val name: String? = null)
```

Get The Passed Arguments

```
1   @Serializable
2   data class Profile(val name: String)
3
4   val navController = rememberNavController()
5
6   NavHost(navController = navController,
7     startDestination = Profile(name="John Smith")) {
8       composable<Profile> { backStackEntry ->
9           val profile: Profile =
10            backStackEntry.toRoute()
11          ProfileScreen(name = profile.name) }
12  }
```

A route with a String parameter

Pass a String value to the Navigation object

NavBackStackEntry object is given as Navigation object param

Get the Profile route

Get the passed name attribute value from the route, and pass it to the actual Profile Screen composable

Demo

Create Serializable Routes

```
1   @Serializable
2   data class Profile(val name: String)
3
4   @Serializable
5   object FriendsList
```

Define ProfileScreen Composable

```
@Composable
fun ProfileScreen(
    profile: Profile
    onNavigateToFriendsList: () -> Unit,
  ) {
  Text("Profile for ${profile.name}")
  Button(onClick = { onNavigateToFriendsList() }) {
    Text("Go to Friends List")
  }
}
```

This Screen forwards the button click event from the inside of Profile screen and directs the navigation route to Friend List screen.

Define FriendListScreen Composable

```
1  @Composable
2  fun FriendsListScreen(onNavigateToProfile: () -> Unit) {
3    Text("Friends List")
4    Button(onClick = { onNavigateToProfile() }) {
5      Text("Go to Profile")
6    }
7  }
```

This Screen forwards “button click” event from the inside of FriendsListScreen and directs the navigation route to Profile screen.

Define MyApp Composable

```
// Define the MyApp composable,
// including the `NavController` and `NavHost`.
@Composable
fun MyApp() {
  val navController = rememberNavController()
  NavHost(navController,
    startDestination = Profile(name = "John Smith")) {
    ... // navigation objects
  }
}
```

This Navigation Host sets the default destination to Profile page and provides a name String as the parameter

Profile Navigation Object

```
1  composable<Profile> { backStackEntry ->
2    val profile: Profile = backStackEntry.toRoute()
3    ProfileScreen(
4      profile = profile,
5      onNavigateToFriendsList = {
6        navController.navigate(route = FriendsList)
7      }
8    )
9  }
```

This Navigation forwards “button click” event from the inside of ProfileScreen and directs the navigation route to FriendsList.

FriendsList Navigation Object

```
composable<FriendsList> {
  FriendsListScreen(
    onNavigateToProfile = {
      navController.navigate(
        route = Profile(name = "Aisha Devi")
      )
    }
  )
}
```

This Navigation forwards “button click” event from the inside of FriendsListScreen and directs the navigation route to Profile while also provides a name String as passed parameter/information

Dialog Destination

Dialog Composable

- To create a dialog destination in Compose, add a destination to the NavHost using the dialog() function.
- The function behaves essentially the same as composable, only it creates a dialog destination rather than a hosted destination.

```
@Serializable
object Settings

// This screen will be
// displayed as a dialog
@Composable
fun SettingsScreen(){
  Card {
    Text("Settings")
    // ...
  }
}
```

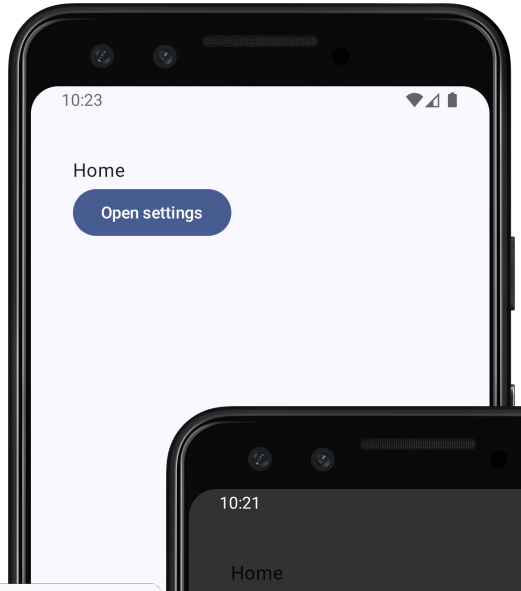
A route to Settings screen

Settings screen composable definition

```
@Composable
fun MyApp() {
  val navController = rememberNavController()
  NavHost(navController, startDestination = Home) {
    composable<Home> { HomeScreen(onNavigateToSettings = {
      navController.navigate(route = Settings) }) }
    dialog<Settings> { SettingsScreen() }
  }
}
```

NavHost with
dialog destination

```
1  @Composable
2  fun HomeScreen(onNavigateToSettings: () -> Unit){
3    Column {
4      Text("Home")
5      Button(onClick = onNavigateToSettings){
6        Text("Open settings")
7      }
8    }
9  }
```




```
1  @Composable
2  fun MyApp() {
3    val navController = rememberNavController()
4    NavHost(navController, startDestination = Home) {
5      composable<Home> { HomeScreen(onNavigateToSettings = {
6        navController.navigate(route = Settings) }) }
7      dialog<Settings> { SettingsScreen() }
8    }
9  }
```

Questions?

Assignments

Assignment

- Berdasarkan assignment sebelumnya, buat satu Home Screen di mana di dalamnya terdapat 4 Button.
- Setiap tombol masing-masing berfungsi untuk mengarahkan tampilan Home Screen ke halaman: Registrasi, Login, Profil User, dan Avatar

Assignment

- Di halaman Profile User, tambahkan satu tombol untuk menampilkan Halaman Avatar dalam bentuk dialog.
- Rekam video yang mendemokan assignment ini dalam durasi maksimal 15 detik.